

Day 106: Logistic Regression

1. Day number

Day 106

2. Topic name

Binary Classification with Logistic Regression

3. Connection to yesterday

Yesterday, you learned **Linear Regression**, which predicts a continuous numerical value.

For example:

- House price → ₹45,00,000
- Temperature → 28.5°C
- Monthly sales → ₹1,20,000

Today, you will learn **Logistic Regression**, which is commonly used for **classification**.

Instead of predicting an exact continuous number, we can predict categories such as:

- Churn → **Yes / No**
- Email → **Spam / Not Spam**
- Loan default → **Yes / No**
- Customer purchase → **Yes / No**

So the main transition is:

Linear Regression → predict a number

Logistic Regression → predict a class

4. Important topics

Classification

Classification means predicting which category an example belongs to.

For example, suppose we want to predict whether a customer will leave a service.

Possible outputs are:

0 = No churn 1 = Churn

The model therefore chooses between categories instead of predicting something like 27.4.

Binary class

Binary classification means there are exactly **two possible classes**.

Examples:

```
0 or 1  
No or Yes  
False or True  
Spam or Not Spam
```

Logistic Regression is especially common for this type of problem.

Probability idea

Logistic Regression can first estimate something that can be interpreted as a **probability for a class**.

For example:

```
Probability of churn = 0.82
```

That means the model considers churn relatively likely for that customer.

A simple rule could then be:

```
Probability >= 0.5 → predict 1  
Probability < 0.5 → predict 0
```

The exact threshold does not always have to be 0.5, but it is a useful beginner default.

Predicted class

After considering the model's probability and classification rule, we obtain the **predicted class**.

Example:

```
Probability of churn = 0.78  
Predicted class = 1
```

If 1 represents churn:

```
Prediction = Churn
```

Logistic Regression

Despite having the word **Regression** in its name, Logistic Regression is mainly used as a **classification algorithm**.

At this stage, you do not need its mathematical derivation. Think of it simply as:

```
Customer features
  ↓
Logistic Regression model
  ↓
Probability
  ↓
Class 0 or Class 1
```

5. Foundational notes

Machine-learning datasets usually contain **features** and a **label**.

Suppose our customer dataset is:

Monthly usage	Support calls	Churn
80	1	0
20	6	1
70	2	0
25	5	1
90	1	0
30	4	1

Here:

```
Features:
monthly_usage
support_calls
```

And:

```
Label:
churn
```

The label contains only:

```
0 = customer stayed
1 = customer left
```

Therefore, this is a **binary classification problem**.

The model attempts to learn relationships between the customer features and the churn label.

6. Regression vs classification

Consider predicting something about a customer.

Regression problem

Question:

| How much money will this customer spend next month?

Possible prediction:

₹4,250

Because the answer is a continuous number, this is **regression**.

Classification problem

Question:

| Will this customer leave the company?

Possible prediction:

Yes

Because the answer belongs to a category, this is **classification**.

A simple rule to remember:

Predict quantity → Regression
Predict category → Classification

7. Easy example — Customer churn

Imagine a subscription company wants to identify customers who may leave.

It collects two features:

monthly_usage
support_calls

Example training data:

Monthly usage	Support calls	Churn
90	1	No
75	2	No
65	2	No
35	4	Yes
25	6	Yes
20	7	Yes

Suppose a new customer has:

```
monthly_usage = 30
support_calls = 5
```

The model might estimate:

```
Churn probability ≈ 0.80
```

Using a simple 0.5 classification threshold:

```
0.80 >= 0.5
```

so the predicted class would be:

```
1 → Churn
```

This is only an illustrative example—the actual probability depends on what the trained model learns from the data.

8. Problem statement

Create a tiny customer dataset containing:

```
monthly_usage
support_calls
churn
```

Use:

```
monthly_usage
support_calls
```

as the **features** and:

```
churn
```

as the **binary label**.

Then:

1. Prepare the features and target.
2. Split the tiny dataset into training and testing data if practical.
3. Create a LogisticRegression model using scikit-learn.
4. Train the model on the training examples.
5. Give the model one new customer.
6. Make one prediction.
7. Interpret 0 as **No Churn** and 1 as **Churn**.

Do not worry about achieving excellent accuracy with such a tiny dataset. The goal is to understand the workflow.

9. Concepts used

This exercise combines several ideas you have already learned:

Feature — information used for prediction.

```
monthly_usage  
support_calls
```

Label / target — the value the model learns to predict.

```
churn
```

Training data — examples shown to the model while learning.

Testing data — examples kept separate to check model behavior.

Model fitting — allowing the Logistic Regression algorithm to learn from the training examples.

Prediction — asking the trained model to classify a new example.

Binary classification — predicting one of two classes.

10. Thought process

Before writing Python, reason through the problem.

First ask:

```
What am I predicting?
```

Answer:

Whether the customer churns.

Then ask:

Is the answer continuous or categorical?

It is categorical:

Yes / No

So this is a **classification problem**.

Next identify the inputs:

monthly_usage
support_calls

And the target:

churn

Then the workflow becomes:

```
Prepare data
↓
Separate features and target
↓
Create train/test sets
↓
Create Logistic Regression model
↓
Fit model
↓
Give model a new customer
↓
Predict class
↓
Interpret 0 or 1
```

11. Beginner-friendly pseudocode

```
START

create a tiny customer dataset

choose:
    monthly_usage
    support_calls
as features X

choose:
    churn
as target y

split X and y into training and testing data

create Logistic Regression model

train model using:
    X_train
    y_train

create one new customer

ask model to predict the customer's class

if prediction is 1:
    display "Churn"
else:
    display "No Churn"

END
```

Notice that this describes the solution without giving you the complete Python implementation.

12. Suggested solving approach — scikit-learn

You will mainly need these scikit-learn ideas:

```
from sklearn.linear_model import LogisticRegression
```

You may also reuse the train/test splitting idea from previous lessons:

```
from sklearn.model_selection import train_test_split
```

The general model workflow is:

```
model = LogisticRegression(...)

model.fit(...)

prediction = model.predict(...)
```

Your job is to decide what belongs inside those ... sections.

For one new customer, remember that scikit-learn normally expects input in a **2D structure** representing:

```
rows = examples
columns = features
```

So conceptually:

```
new customer
    ↓
[[monthly_usage, support_calls]]
```

rather than simply treating the two numbers as unrelated values.

13. Easy edge cases

Only one class exists

Suppose your training data accidentally contains:

```
0
0
0
0
0
```

There are no examples of class 1.

A Logistic Regression classifier cannot meaningfully learn to distinguish two classes when its training data contains only one class, and scikit-learn will normally raise an error.

Make sure your tiny training dataset contains examples from **both classes**.

Unusual input

Suppose the model has only seen monthly usage between:

```
20 and 100
```

but you ask it to predict:

```
monthly_usage = 10000
```

The model can still attempt a prediction, but the input is far outside the examples it learned from.

Treat such predictions cautiously.

This illustrates an important ML lesson:

A model usually works best when new data resembles the type of data on which it was trained.

14. Expected prediction

Imagine the new customer is:

```
monthly_usage = 28  
support_calls = 6
```

After training, suppose:

```
model.predict(...)
```

returns:

```
[1]
```

Interpretation:

```
1 = Churn
```

So you could describe the result as:

```
Predicted class: Churn
```

Another customer could instead produce:

```
[0]
```

meaning:

```
Predicted class: No Churn
```

Your exact result can differ depending on your tiny dataset and train/test split.

15. Hint only

Build something approximately like this structure:

```
tiny DataFrame
  ↓
X = feature columns
y = churn column
  ↓
train_test_split(...)
  ↓
LogisticRegression(...)
  ↓
model.fit(?, ?)
  ↓
new_customer = [[?, ?]]
  ↓
model.predict(?)
```

Fill in the missing pieces yourself.

The most important idea from **Day 106** is:

Logistic Regression learns from features to classify an example into one of two classes, often using an estimated probability before producing the final class prediction.

Tomorrow's classification topics will be much easier if you are comfortable identifying **features, binary labels, probabilities, and predicted classes**.